2024 IEEE International Conference on Software Maintenance and Evolution (ICSME) | 979-8-3503-9568-6/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICSME58944.2024.00090

GitTruck@Duck – Interactive Time Range Selection in Hierarchy-Oriented Polymetric Visualization of Git Repository Evolution

Adrian Hoff*, Thomas Hoffmann Kilbak*, Leonel Merino†, Mircea Lungu*,
*IT University of Copenhagen, Denmark
†School of Design, School of Engineering, Pontificia Universidad Católica de Chile

***Abstract*—In this work, we present GitTruck@Duck; a web-powered software visualization tool that combines hierarchical file structure with configurable software evolution and collaboration metrics on adjustable time ranges. In an automated mining process, Git Truck aggregates a system's Git history using an in-memory relational database, along with an algorithm for detecting the renaming of files over time. Users can explore the Git history in a 2D hierarchical visualization of folders and files, where they encode evolution and collaboration metrics (e.g., top contributors or last change date) on the size and color of the marks that represent files. Users can gain fine-grain control over these metrics by specifying the time ranges for inspection.**

***Index Terms*—Software Engineering, Software Visualization, Software Evolution, Mining Software Repository, Git**

***Demo Video*—https://youtu.be/StXvpVJUK1I**

I. Introduction

Significant progress has been made in software visualization to analyze software evolution, particularly, regarding changes in source code and their impacts [1], [2]. In effect, there are numerous tools to aid in this analysis, focusing primarily on changes in structural metrics over time (such as the size of software building blocks). However, they often miss important details that help engineers with understanding the evolution of a system: *Who contributed to which part of the system? Which files were changed recently? Which parts of the system were changed drastically?*

In our previous work, we introduced Git Truck, a tool designed to automatically analyze Git source code repository histories and provide an interactive polymetric 2D visualization [3], [4]. Git Truck helps answer questions about developer contributions, recent changes, and significant modifications, among others. Despite its capabilities, Git Truck, like many other tools, offers only visualizations for the entire history of a Git repository. This limitation prevents users from querying specific time ranges, which is essential for understanding fine-grained temporal aspects such as changes in code ownership.

II. Using GitTruck@Duck

We present GitTruck@Duck, a new release of Git Truck that enables users to filter and display polymetric views scoped to specific time intervals that the users can adjust in real-time in the tool's UI. A central aspect in GitTruck@Duck is its responsiveness and efficiency so that users can dynamically analyze large repositories by continuously querying new data and changing views without compromising tool performance. This allows for more nuanced investigations into the evolution of a repository and the associated developer contributions.

We illustrate the UI of GitTruck@Duck with an example of the Linux repository[1], briefly pointing out how the time-range selection can highlight the evolution of developer activity.

A. Hierarchical File Visualization

Figure 1 shows two different screenshots of GitTruck@Duck visualizing the `/kernel` folder[2] from the Linux repository. The core of the app is a hierarchical file visualization – the colorful main areas in the screenshots. It displays all files, sub-folders (and their content recursively) within a given folder in the Git repository's main branch. Users can navigate to a sub-folder by clicking on it, which updates the display to show only the selected folder. In the figure, the user has selected the `./kernel` folder.

B. Polymetric Views

Users control the metrics that are mapped on the size and color of file tiles in the main view via a configuration side panel located on the left-hand side in the UI (see Figure 1). The configuration side panel also allows users to choose the hierarchical layout: either a *bubble chart* or a *tree map*.

In Figure 1, GitTruck@Duck scales the tiles based on the number of commits and colors them according to the top contributor. In the contributor panel (bottom of the configuration panel), Linus Torvalds was manually assigned the color black.

C. The Time Range Selector

In the bottom panel, users scope the time interval for analysis by adjusting a time range, either by dragging the sliders or by clicking on the date labels to specify a particular date which opens a mini-calendar. The time range slider is accompanied by an activity bar chart, showing the number of commits that occurred during each interval.

If one scopes the analysis for the early history of the Linux project, e.g., from its initialization on April 17, 2005, to January 1, 2008, the visualization reveals that Linus Torvalds (black tiles) was the most active developer during this period.

[1]https://github.com/torvalds/linux

[2]We start with the already focused /kernel folder in the paper because the full repository view results in details that are barely visible in print resolution

2576-3148/24/$31.00 ©2024 IEEE
DOI 10.1109/ICSME58944.2024.00090

Authorized licensed use limited to: Mara Ielciu. Downloaded on March 15,2026 at 22:19:41 UTC from IEEE Xplore. Restrictions apply.

Fig. 1. Screenshots of GitTruck@Duck visualizing the Linux kernel. Each tile in the colored treemap represents a file. The tile size represents the number of commits and the colors map to the top contributor. A slider (bottom area) allows for the selection of time ranges for which the above metrics are shown. The screenshot to the left shows top contributors in the early years of the Git repository, while the right-hand screenshot (side panel omitted) shows more recent top contributors.

Adjusting the time range to span from 2008 to 2024, results in the updated visualization on the right-hand side of Figure 1. This new perspective shows no more black tiles: it seems that other contributors have taken over Linus Torvalds' dominant role in developing the Linux kernel, as he does not appear as the top contributor in this more recent period.

D. Setting up GitTruck@Duck

To use GitTruck@Duck, users need to install Node.js[3] and Git[4]. With these tools installed, GitTruck@Duck is installed by running `npm install -g git-truck@duck`. Alternatively, using the `npx` utility, GitTruck@Duck can be downloaded and started with a single command by navigating to a folder that contains cloned Git repositories (irrespective of the specific Git host) and running `npx -y git-truck@duck`. This initiates GitTruck@Duck's automated Git repository history analysis and automatically opens the results in the default web browser. More details and up-to-date information are available in the GitHub repository for GitTruck@Duck[5].

III. Tool Implementation

Figure 2 shows the overall architecture of GitTruck@Duck. The tool is implemented as a web application using Node.js and the Remix full-stack framework. The core of GitTruck@Duck is its backend which is responsible for (a) performing the analysis of Git repository histories by communicating with a local Git process and (b) providing users with a user interface for exploring the analysis results through a configurable visualization in their web browser.

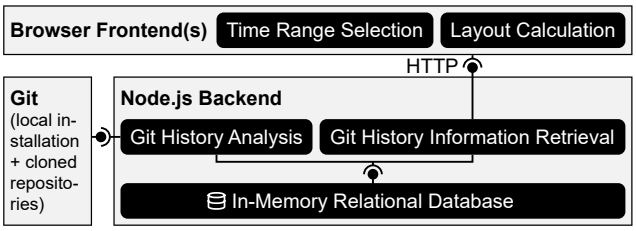

Fig. 2. Architecture of GitTruck@Duck. A Node.js backend performs the analysis by communicating with a Git process and storing information in a database (locally, on the user's machine). Users then access a web-powered interface frontend to inspect and interact with the visualization (see Figure 1).

A. Considerations for Tool Performance

A central aspect in the technical implementation of GitTruck@Duck is achieving efficient data handling. This concerns three main aspects, discussed in the following.

1) Efficient Data Aggregation: The backend implementation must efficiently aggregate data to generate polymetric views, such as determining commit counts per file, for color representation and ensuring that updates are timely during user interactions with time ranges. To achieve this, given the scale of the data involved, we opted for DuckDB [5], an in-memory database optimized for analytics. The columnar

[3]https://nodejs.org/en
[4]https://git-scm.com/downloads
[5]**https://github.com/git-truck/git-truck**

Authorized licensed use limited to: Mara Ielciu. Downloaded on March 15,2026 at 22:19:41 UTC from IEEE Xplore. Restrictions apply.

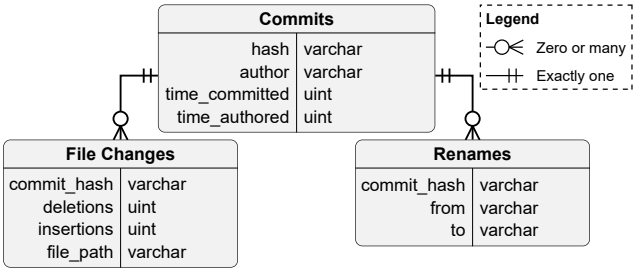

Fig. 3. Entity-relationship diagram of the database schema employed in GitTruck@Duck. It models Git repository histories as individual commits with associated file changes and rename operations.

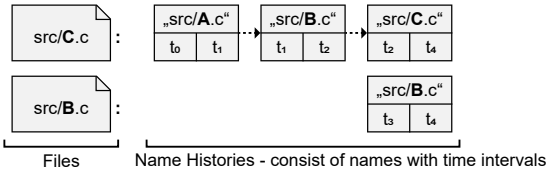

Fig. 4. Name histories per file as extracted by GitTruck@Duck based on file creation and rename information provided by Git.

architecture of DuckDB is beneficial for aggregating data since only necessary columns are loaded —unlike row-store databases such as SQLite, where entire tables must be loaded for data aggregation purposes. We compare the performance of SQLite and DuckDB for typical GitTruck@Duck queries in Section IV.

Another option for data management is graph databases. Although these would offer a more accurate model of Git histories by following their commit graph, using a graph database would likely incur performance problems, e.g. when retrieving all descendants of a commit [6].

2) Efficient Time Range Querying: The backend implementation must be able to efficiently retrieve commits within specified time ranges without necessitating full data scans. For this purpose, choosing a relational database facilitates efficient querying within specified ranges, leveraging indexing on relevant columns to avoid full table scans.

3) Efficient Data Insertion: The backend implementation must efficiently handle the insertion of large amounts of Git metadata to facilitate swift initial analysis, with less emphasis on small transaction performance once the initial setup is complete. To address this, we utilize DuckDB's specialization in bulk data ingestion to batch-insert commits, and thus enhance efficiency. Managing potentially large volumes of commits in Node.js could surpass its default heap size, hence transferring batches to DuckDB, where data can be compressed and optionally stored on disk, resolves this issue effectively. This strategy involves inserting batches of several thousand rows at a time to balance performance and memory usage.

B. Database Model

Figure 3 shows the database model used in GitTruck@Duck. It is based on the data model used in the Gitana tool [7] but has been significantly simplified to store only essential information required for calculating metrics currently used in GitTruck@Duck's polymetric visualization views. Notably, commit messages and descriptions are omitted from the database to minimize memory consumption, as they are unnecessary for GitTruck@Duck's metric generation and can be lazily loaded on demand, i.e. when a user selects a file.

In the Commits table, both the *commit time* and *author time* are stored to handle cases where commits share the same commit time, typically encountered during history rewrites such as rebasing. This distinction allows GitTruck@Duck to sequence commits based on their author timestamps in scenarios where commit times are identical. The `Renames` table records file renaming events in commits, storing the original and new file names along with the timestamp of each rename.

C. Tracking File Renames in Git Histories

A major challenge in the development of functionality similar to that presented in this work is accurately tracking file renames throughout Git histories. To calculate correct values for evolution metrics, it is crucial to ensure that commits to old file versions are correctly attributed and not counted for new files created with the same name later. While Git offers the `git log --follow` command to track file renames, this approach has significant limitations. It operates on a per-file basis and does not support querying the entire repository's commit history while following renames. Thus, the command becomes inefficient for large repositories as each commit must be analyzed separately for every file it affects.

To address this, we extract occurrences of file creation and renaming from Git directly and persist them in the database, associated with a hash of the respective commit as seen in the `Renames` table shown in Figure 3. Based on that, we go through the repository history in reverse order and aggregate rename occurrences (including file creations) by joining the tables `Renames` and `Commits`. We use this information to iteratively determine the name history for each file as a chained list of names attributed with the time interval in which these apply[6]. Figure 4 illustrates that structure conceptually. It allows GitTruck@Duck to track files across time.

With every time range selection change in the user interface, GitTruck@Duck creates a *rename-aware time-range-specific temporary table* which is a join of the tables presented in Figure 3 and the name histories presented in Figure 4. This temporary table contains all the information required to update the visualization based on parameters from the configuration side panel, effectively reducing respective update times.

D. Limitations to the Approach

While the described data model enables efficient Git history processing, it has accuracy limitations in certain edge cases.

The first limitation is that GitTruck@Duck relies on Git's built-in rename detection algorithm, which may fail if a

[6]More details: https://www.doi.org/10.6084/m9.figshare.26763283

Authorized licensed use limited to: Mara Ielciu. Downloaded on March 15,2026 at 22:19:41 UTC from IEEE Xplore. Restrictions apply.

file undergoes significant changes during renaming [8]. Our subsequent analysis inherits these potential inaccuracies.

The second limitation arises from using commit timestamps to determine the sequence of commits. Since a Git history forms a graph rather than a linear sequence, this method can lead to inaccuracies. That is, with our data model, we assume that the files are not renamed in one branch and later modified in its previous version on another branch before merging. If this assumption is violated, line changes that should have been counted towards the total for the given file or contributing author will be lost. Achieving complete correctness would require traversing the Git graph to gather commit statistics based on parent relationships rather than timestamps. However, this approach would likely reduce performance in generating aggregate statistics given that our in-memory model is stored in relational tables.

Existing work on concurrency and conflicts in Git repositories shows that for the Linux project, 10. 99% of file changes occurred to files on parallel branches, of which 4.86% resulted in an unresolved conflict [9]. Of these, 9% are conflicts in removal or naming, so that, in total, at most 0.05% of file changes in the Linux could have been involved in the scenario presented above. Although for use cases like Git-Truck, where the focus is on providing a high-level overview of a repository's history, trading precision for speed is acceptable this might not apply to other tools. Future work could also explore improving correctness, detecting problematic commit histories, or advising users when such cases occur.

IV. Performance Evaluation

Next, we show the outcomes of the performance benchmarks related to the design choices for efficient data handling, as discussed in the preceding section. All benchmarks were run on the same laptop (CPU Ryzen 7 4800U, Hard drive 1 TB NVMe SSD, OS Fedora 40, Kernel Linux 6.8.9-300) using Node.js v21.1.0, DuckDB v0.10.2. All tests were run twice to check that the results are consistent.

A. DuckDB vs SQLite

Figure 5 presents the execution times for running three types of queries frequently executed by GitTruck@Duck. That is, (1) a query performing an expensive *table join* (described in Section III-C), (2) a query to determine the top contributors per file, and (3) a query to count the number of commits per file (see the visualized metrics in Figure 1). The results show that DuckDB executed all queries in significantly less time compared to SQLite – which is why we chose DuckDB as the database solution for GitTruck@Duck.

B. User Interaction

The analysis presented in Section III scales primarily with the number of commits pushed to a subject Git repository. Thus, we evaluate GitTruck@Duck's performance by selecting four Git repositories with varying numbers of commits and measuring the worst-case delays in its user interface. Figure 6 plots the results of that in a linear scale. The measured

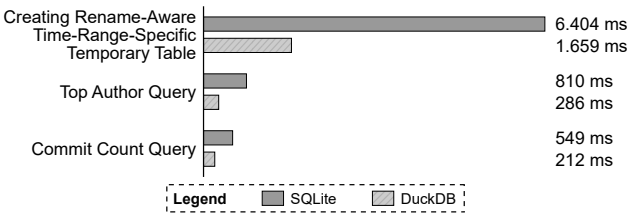

Fig. 5. Comparison of the execution duration for running queries using DuckDB (v0.10.2) and SQLite (v5.1.1).

delays occur when users perform changes to the hierarchical visualization (e.g., switching between metrics mapped to the size and color of visual elements) while having selected the subject system's entire Git history as current time range.

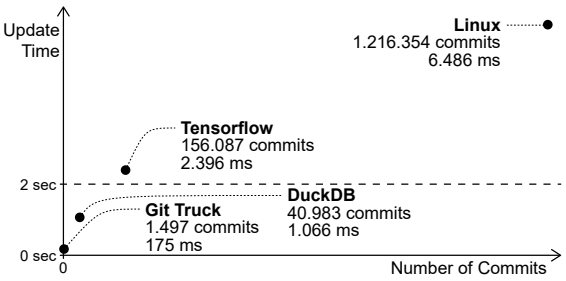

Fig. 6. Worst-case user interface update times when changing the view in GitTruck@Duck during an analysis of repositories with varying numbers of commits. The dashed line marks a delay of two seconds.

As a reference for an acceptable delay in the tool's user interface, we use two seconds [10]. The presented results show that the threshold of two seconds is exceeded for repositories with a larger number of commits (e.g., Tensorflow with 156.087 commits at the time of writing this paper). However, the results also show that the worst-case delays in Git Truck's user interface exhibit sublinear growth – a growing number of commits causes more worst-case delay, but the additional increments in time become smaller.

V. Related Work

The related work falls into two areas: visualization of hierarchical file structures and the evolution of software systems.

a) Hierarchical File Structure Visualizations: There are multiple tools to visualize the file and folder structure of a software system using hierarchical views [11]. These can be distinguished by spatial dimensions (2D and 3D), layout algorithms (e.g., "squarified" or circular treemap layouts as employed by GitTruck@Duck), or their usage of edges between visual elements [12]. In contrast to many tools that focus on structural aspects of software systems [13]–[16], GitTruck@Duck visualizes metrics on their evolution and contributor collaboration.

b) Software Evolution Visualizations: Various tools exist to visualize the evolution of software systems. Many of these provide their users with visual summaries over fixed time ranges of a repository's history (typically the entire

Authorized licensed use limited to: Mara Ielciu. Downloaded on March 15,2026 at 22:19:41 UTC from IEEE Xplore. Restrictions apply.

history [17] or one commit at a time [18]–[21]). In contrast to these tools, our extension to GitTruck@Duck provides users with a means for filtering evolution metrics by specific time ranges of interest. Other software evolution visualization tools provide users with means to select time ranges that should be visualized, e.g., by summarizing clusters of coherent subsequent commits [22] or by providing users with interfaces for direct specification of time ranges (such as the slider in GitTruck@Duck) [23], [24]. However, these commonly focus on change-related metrics such as the size of elements over time or quantification of file change. In contrast, GitTruck@Duck provides users with metrics that foster the analysis of contributor collaboration in a Git repository.

VI. Conclusion and Future Work

In this article, we introduced GitTruck@Duck, an extension to our previous Git Truck project, which allows users to explore file history within a Git repository. GitTruck@Duck is a platform-independent, easy-to-install visualization tool. Users inspect and configure GitTruck@Duck's visualization via an interface in their web browser. Our tool combines the hierarchical file structure with configurable software evolution and collaboration metrics over flexible time intervals. The design of GitTruck@Duck focuses on efficient data handling, ensuring that performance remains uncompromised as users continuously query new data and change views while analyzing repositories. We demonstrate the tool by exploring the Linux kernel repository. Subsequently, we assessed the performance of GitTruck@Duck. Our findings indicated that DuckDB executed three types of queries markedly faster than SQLite. We examined the maximum delays in users' interactions, revealing that the tool incurs a delay of up to two seconds solely when processing extensive commits (exceeding 156,087 commits), demonstrating sublinear growth.

In future work, we will enhance GitTruck@Duck's data management backend to follow a hybrid approach combining relational and graph databases to enhance performance and correctness checks. Further, we plan to conduct user studies, i.e., a long-term study to evaluate the tool's utility in larger repositories and various team contexts, and focused small-scale studies to test specific hypotheses, such as the effectiveness of time sliders for onboarding new developers.

Acknowledgments

Hoff has been supported by the DFF (Independent Research Fund Denmark) within the project ISA – Immersive Software Archaeology (0136-00070B). Merino has been supported by ANID FONDECYT Iniciación Folio 11230349.

References

[1] R. L. Novais, A. Torres, T. S. Mendes, M. Mendonça, and N. Zazworka, "Software evolution visualization: A systematic mapping study," *Information and Software Technology*, vol. 55, 2013.

[2] A.-L. Mattila, P. Ihantola, T. Kilamo, A. Luoto, M. Nurminen, and H. Väätäjä, "Software visualization today: systematic literature review," in *Proceedings of the 20th International Academic Mindtrek Conference*. ACM, 2016.

[3] K. Hojelse, T. Kilbak, J. Rossum, E. Japelt, L. Merino, and M. Lungu, "Git-Truck: Hierarchy-Oriented Visualization of Git Repository Evolution," in *2022 Working Conference on Software Visualization (VISSOFT)*. IEEE, 2022.

[4] M. Lungu, R.-H. Pfeiffer, M. D'Ambros, M. Lanza, and J. Findahl, "Can Git Repository Visualization Support Educators in Assessing Group Projects?" in *2022 Working Conference on Software Visualization (VISSOFT)*. IEEE, 2022.

[5] M. Raasveldt and H. Mühleisen, "DuckDB: an Embeddable Analytical Database," in *Proceedings of the 2019 International Conference on Management of Data*. ACM, 2019.

[6] T. A. Falcão, P. M. Furtado, J. S. Queiroz, P. J. O. Matos, T. F. Antunes, F. S. Carvalho, P. C. R. Fonseca, and F. T. Giuntini, "Comparative Analysis of Graph Databases for Git Data," *Journal of Physics: Conference Series*, vol. 1944, 2021.

[7] V. Cosentino, J. L. C. Izquierdo, and J. Cabot, "Gitana: A SQL-Based Git Repository Inspector," in *Conceptual Modeling*. Springer International Publishing, 2015, vol. 9381.

[8] A. Fujimoto, Y. Higo, and S. Kusumoto, "Towards Accurate File Tracking Based on AST Differences," in *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2021.

[9] H. Le Nguyen and C.-L. Ignat, "An Analysis of Merge Conflicts and Resolutions in Git-based Open Source Projects," *Computer Supported Cooperative Work*, vol. 27, 2018.

[10] F. F.-H. Nah, "A study on tolerable waiting time: how long are Web users willing to wait?" *Behaviour & Information Technology*, vol. 23, 2004.

[11] L. Merino, E. Kozlova, O. Nierstrasz, and D. Weiskopf, "VISON: An ontology-based approach for software visualization tool discoverability," in *2019 Working Conference on Software Visualization (VISSOFT)*. IEEE, 2019, pp. 45–55.

[12] H.-J. Schulz, S. Hadlak, and H. Schumann, "The Design Space of Implicit Hierarchy Visualization: A Survey," *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, 2011.

[13] N. Anquetil, A. Etien, G. Andreo, and S. Ducasse, "Decomposing God Classes at Siemens," in *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2019.

[14] G. d. F. Carneiro, C. Sant'Anna, A. Garcia, C. Chavez, and M. Mendonça, "On the use of software visualization to support concern modularization analysis," *Contemporary Modularization Techniques (ACoM. 08)*, 2009.

[15] C. Li, Y. Pei, Y. Shen, J. Lu, Y. Fan, X. Linghu, Y. Tian, and K. Wang, "PyVisVue3D3: Python visualization from hierarchy tree to call graph," *SoftwareX*, vol. 26, 2024.

[16] I. Bacher, B. M. Namee, and J. D. Kelleher, "On Using Tree Visualisation Techniques to Support Source Code Comprehension," in *2016 IEEE Working Conference on Software Visualization (VISSOFT)*. IEEE, 2016.

[17] H. Tarner, V. Frick, and M. Pinzger, "Visualizing Evolution and Performance Metrics on Method Level as Multivariate Data," *SATToSE*, 2020.

[18] A. McNair, D. M. German, and J. Weber-Jahnke, "Visualizing Software Architecture Evolution Using Change-Sets," in *14th Working Conference on Reverse Engineering (WCRE 2007)*. IEEE, 2007.

[19] M. D'Ambros, M. Lanza, and M. Lungu, "Visualizing Co-Change Information with the Evolution Radar," *IEEE Transactions on Software Engineering*, vol. 35, 2009.

[20] W. Scheibel, J. Blum, F. Lauterbach, D. Atzberger, and J. Döllner, "Integrated Visual Software Analytics on the GitHub Platform," *Computers*, vol. 13, 2024.

[21] S. Rufiange and G. Melançon, "AniMatrix: A Matrix-Based Visualization of Software Evolution," in *2nd IEEE Working Conference on Software Visualization, VISSOFT 2014*. IEEE, 2014.

[22] Y. Kim, J. Kim, H. Jeon, Y.-H. Kim, H. Song, B. Kim, and J. Seo, "Githru: Visual Analytics for Understanding Software Development History Through Git Metadata Analysis," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, 2021.

[23] T. Dal Sasso, R. Minelli, A. Mocci, and M. Lanza, "Blended, not stirred: Multi-concern visualization of large software systems," in *2015 IEEE 3rd Working Conference on Software Visualization (VISSOFT)*. IEEE, 2015.

[24] C. V. Alexandru, S. Proksch, P. Behnamghader, and H. C. Gall, "EvoClocks: Software Evolution at a Glance," in *2019 Working Conference on Software Visualization (VISSOFT)*. IEEE, 2019.

Authorized licensed use limited to: Mara Ielciu. Downloaded on March 15,2026 at 22:19:41 UTC from IEEE Xplore. Restrictions apply.